Indexes, Constraints, and Performance Notes

Introduction

Indexes and constraints are critical elements in **database design and performance tuning**. They ensure data integrity, enforce business rules, and optimize query performance.

In Business Central and Azure SQL environments, these elements help maintain **scalability**, **accuracy**, and **speed** when handling large transactional datasets like `SalesHeader`, `SalesLine`, and `Warehouse Shipment`.

Key purposes:

- **Indexes** improve search, filtering, and join performance.
- **Constraints** ensure data quality and enforce relationships.
- **Performance notes** guide future maintenance and monitoring practices.

Diagram Components

Component	Description
Index (Blue Box)	Speeds up query performance for specific fields such as `Document No.` or `Customer No.`
Constraint (Green Box)	Enforces data rules, e.g., `FOREIGN KEY`, `PRIMARY KEY`, `CHECK`
Performance Callout (Yellow Box)	Notes or tips for tuning database behavior and troubleshooting
Transactional Table	Core tables like `SalesHeader` and `SalesLine` that benefit from optimization

---

Modeling Symbol Definitions

Symbol	Representation
Blue Cylinder	Index object

Green Key Icon	Constraint (Primary or Foreign Key)
Yellow Sticky Note	Performance best practice or monitoring note
Solid Line →	Index applied to a single field
Dashed Line →	Constraint applied across multiple fields

Common Business Central Examples

Object Type	Table Example	Field Example	Purpose
Index	`SalesHeader`	`Document No.`	Speeds up document searches
Index	`SalesLine`	`Item No.` + `Location Code`	Optimizes filtering for picking
Constraint - PK	`Customer`	`Customer No.`	Unique identifier for customer records
Constraint - FK	`SalesHeader` → `Customer`	`Customer No.`	Ensures valid customer exists
Constraint - Check	`Currency`	`Exchange Rate > 0`	Prevents invalid exchange rates

Performance Notes

- Regularly **rebuild or reorganize indexes** to prevent fragmentation.
- Avoid creating too many indexes, which can slow down `INSERT` and `UPDATE` operations.
- Use **non-clustered indexes** for reporting tables.
- Monitor query performance with **SQL Server Query Store**.
- Apply constraints to **prevent data corruption**, not just for documentation.
- Consider **partitioning large transactional tables** like `SalesLine` or `ValueEntry` for scalability.

Diagram Example

The visual will show:

- `SalesHeader` and `SalesLine` tables with **blue index indicators** on common fields.
- **Constraints** showing primary keys and foreign key relationships.
- **Callouts** with optimization notes for future tuning.

Indexes, Constraints, and Performance Notes

Introduction

Indexes and Constraints icss ass essential mechanisms for ensuring data integrity and optimizing query performance. While constraints enforce rules such as unique values, and safe deletion data.

- Indexes: represent table indites for query performance.
- Two types of indexes: Clustered– which defines the physical data storage and Non-clustered- which is a separate structure pointing to the data.
- Constraints: applied to table columns to enforce data integrity rules

Modeling Symbol Definitions

- Table: represents an atable
- PK Clustered index
- PK Performance notes

Diagram Components

- Table: Represents table indices
- PK Constraint: represents rules for data integrity
- PK Performance notes: contains comment-like annotations
- Rectangle a Table
- Clustered index
- Non-clusterded index

Diagram Example

- Indexes; representing a table indices
- Constraints; representing rules for data integrity
- Performance notes: containing comment-like annotations
- **Note:** Stored for erformance-related comments or optimiza-tion hints